GRC102: Information Security Governance Practical Lab

Security Governance Simulation, Control Implementation and Executive Reporting

Item	Detail
Student	Chioma Ameh
Student ID	C11/26/CGRCE/17567
Date submitted	11 October 2026
GitHub repository	https://github.com/ChiomaAmeh/grc102-lab8-security-governance
Branch (latest, includes this report)	https://github.com/ChiomaAmeh/grc102-lab8-security-governance/tree/main
Commit with all scripts and outputs	https://github.com/ChiomaAmeh/grc102-lab8-security-governance/commit/b71d405
Environment	Kali Linux (kernel 6.16.8) in a VirtualBox VM on Windows 11; Docker 28.5.2; Docker Compose 2.40.3
Time zone note	The VM clock runs on US Eastern Time (EDT, UTC-4). West Africa Time (WAT) is 5 hours ahead, so 07:34 EDT in a screenshot is 12:34 WAT.
Scope	Isolated lab only. Synthetic data only. No external systems were scanned or tested.

Executive summary

I built the instructor's Docker simulation of an Equifax-style environment: a web server running Apache Struts 2.3.30 (CVE-2017-5638), a MySQL 5.7 database with default credentials, an idle monitoring container and a flat network. The official scanner reported **Critical 1, High 1, Medium 1**. A simulated attack succeeded, and the governance analysis identified **8 failures**. The official control script then recorded five controls, but the verification step showed they were **not operating**: the re-scan still reported Critical 1, High 1, Medium 1 and the attack still succeeded, because the official patch commands only write a status file. I then applied a clearly labelled supplementary real remediation. After it, the scan showed **Critical 0, High 0, Medium 1** and the attack simulation reported the system not vulnerable. The remaining Medium finding is a limitation of the official segmentation check, and I proved the segmentation manually. Finally, I found that the official dashboard and board report use random and hard-coded values that contradict the measured evidence, so I report the real measurements alongside them.

Contents

- Part 1: Simulated Environment
- Part 2: Governance Failure Analysis
- Part 3: Governance Control Implementation
- Part 4: Metrics and Executive Reporting
- Conclusion and limitations
- References

- Appendix A: Troubleshooting log and deviations
- Appendix B: Repository file inventory

Part 1: Simulated Environment

1.1 Design and isolation

Following Task 1, I created a Docker network `secgov_network` and a `docker-compose.yml` with three containers: `web_server` (vulhub Struts 2.3.30), `database_server` (MySQL 5.7 loaded with three synthetic customer records from `database_init/init.sql`) and `monitoring_server` (Ubuntu 20.04, idle). All data is synthetic; no real personal data, passwords or external targets were used. The only published port was 8080 on the lab VM. The lab runs in an Oracle VirtualBox VM on a Windows 11 laptop, and no other host was scanned or tested. [VirtualBox network mode: _____ (Settings > Network > Adapter 1 > Attached to)] After remediation I rebound the port to 127.0.0.1 only.

Weakness built into the lab	How it appears in the lab	Equifax parallel
Unpatched known flaw	Apache Struts 2.3.30 with CVE-2017-5638 (S2-045); fixed in 2.3.32 / 2.5.10.1	Known Struts flaw left unpatched after a fix was available
Weak authentication	MySQL root password is "password"	Weak credentials and access controls
Flat network	Web and database share one network	Attackers moved laterally
No effective monitoring	monitoring_server only runs `tail -f /dev/null`	Breach undetected for about 76 days
Policies without enforcement	Tracker holds a patch policy, yet patch compliance is 85% against 95%	Policies existed but were not enforced

1.2 Building the environment (Task 1)

I installed Docker and Compose on Kali, created the network, the compose file, the database initialisation file and the monitoring folder, and started the containers. Two problems occurred and are explained in Appendix A: Docker was not installed, and then my user was not in the docker group (permission denied on the Docker socket). Later, `docker-compose up -d` failed with "manifest unknown" for the image tag `s2-045`; the same image was already stored locally as 2.3.30, so I gave it the official name with `docker tag` and the instructor's compose file worked unchanged.

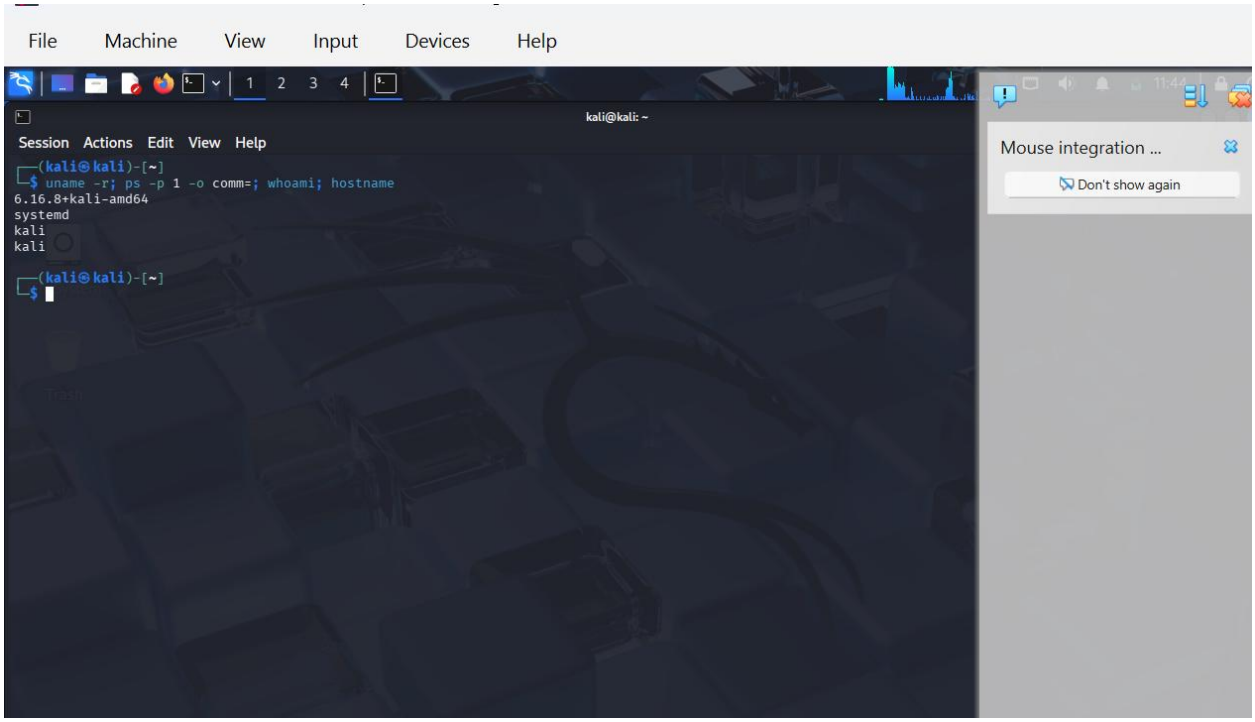


Figure 1.1 - Kali Linux VM with systemd (kernel 6.16.8); user kali

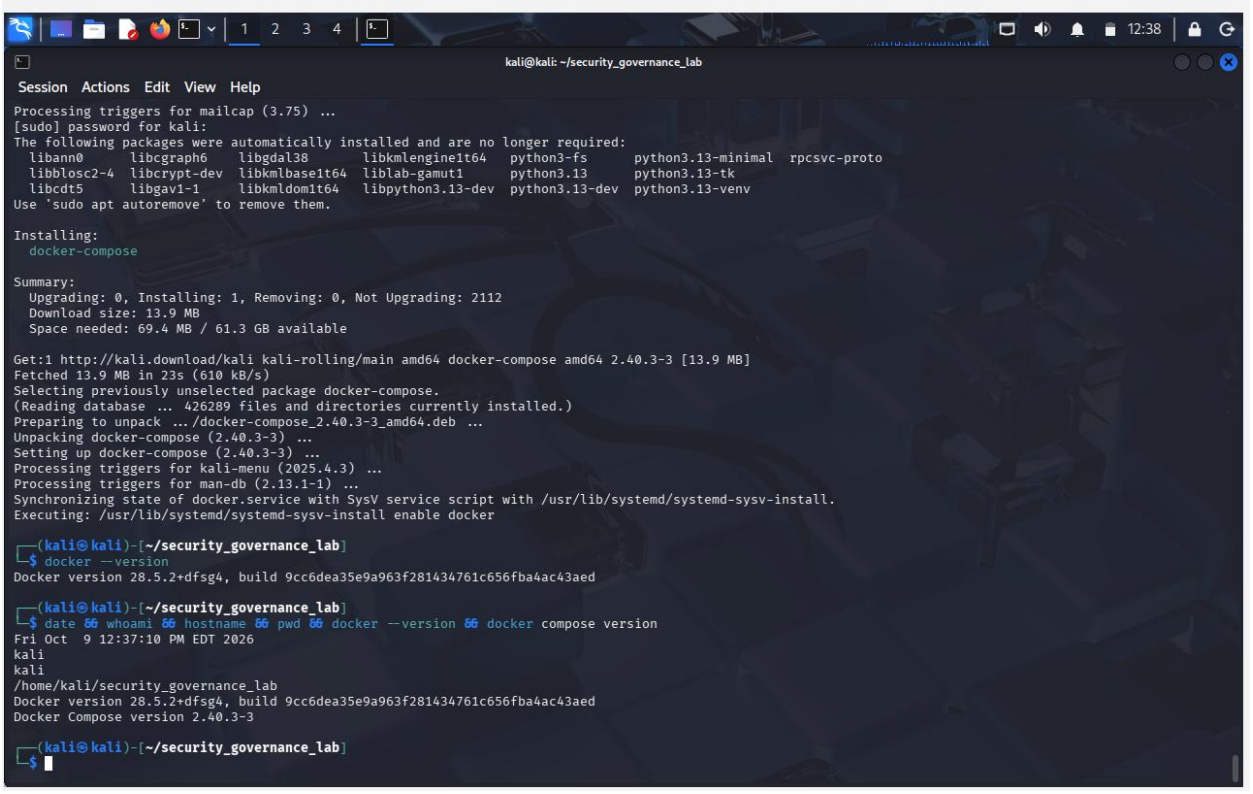


Figure 1.2 - Docker 28.5.2 and Compose 2.40.3 installed; working directory and date

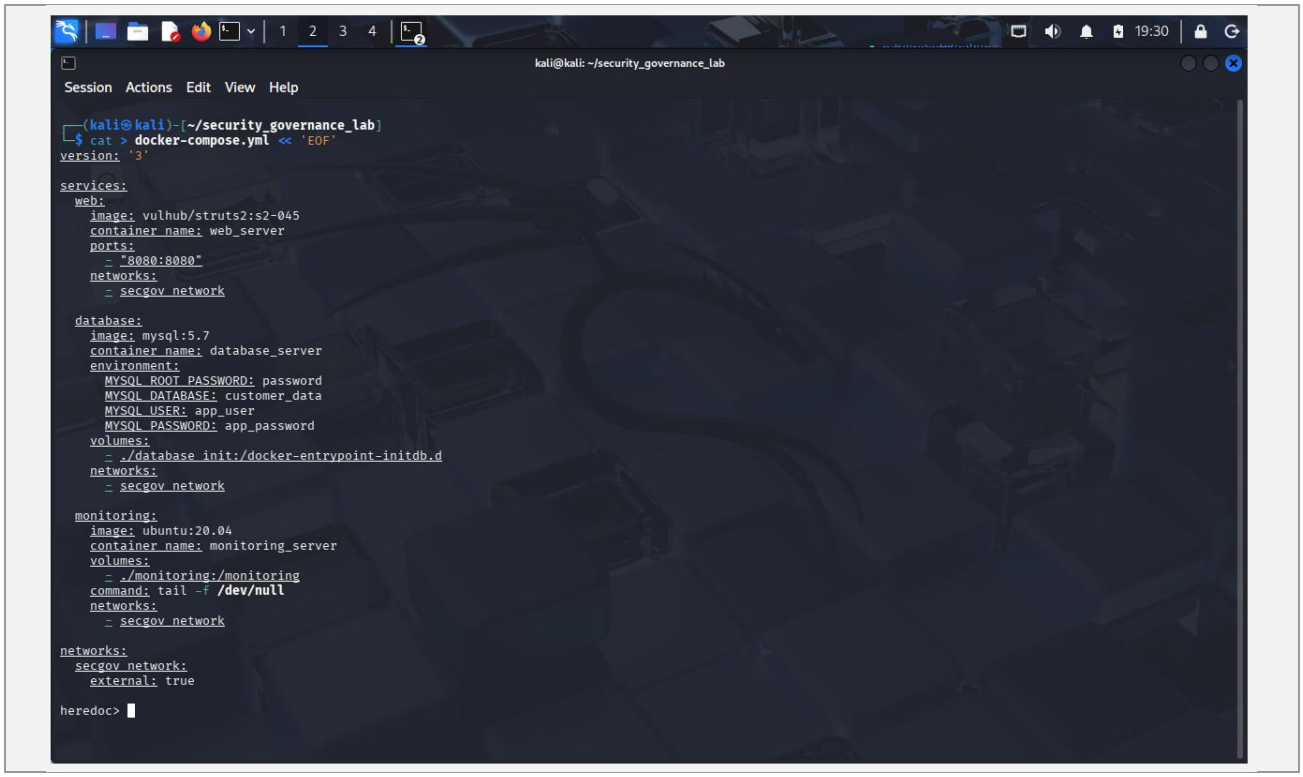
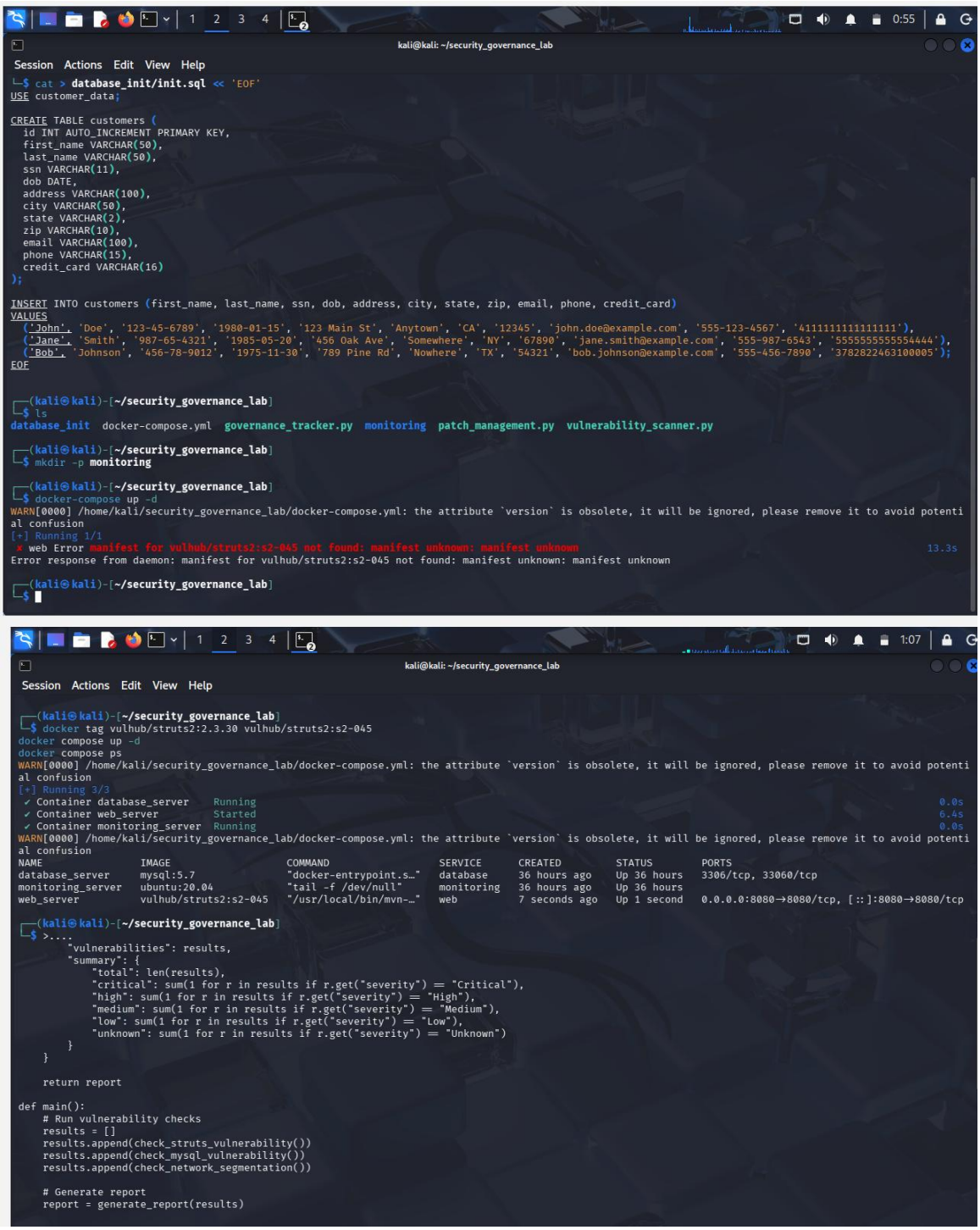


Figure 1.3 - docker-compose.yml created from the instructor's Task 1 (three services and the secgov_network network)



Figures 1.4 and 1.5 - Top: database initialisation file created, then docker-compose up fails with "manifest unknown" for s2-045 (troubleshooting). Bottom: fix with docker tag (same image, official name); three containers running

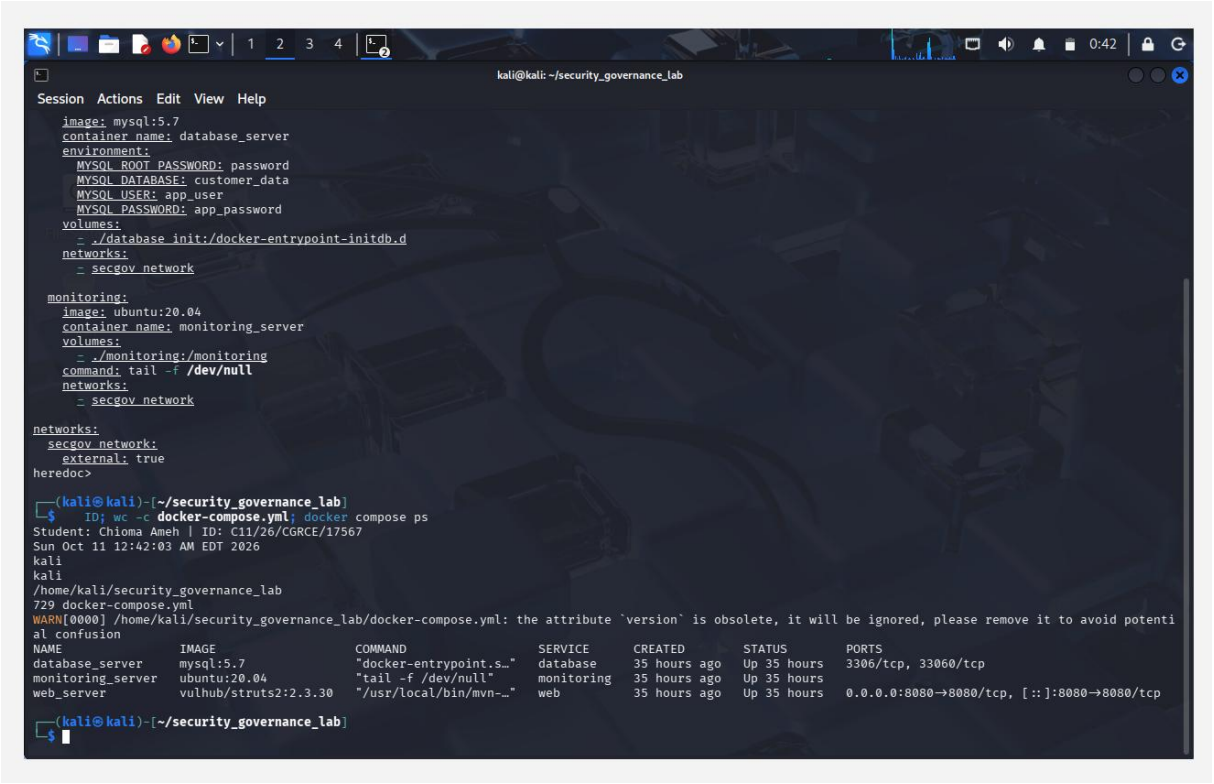


Figure 1.6 - Three containers up (web, database, monitoring) with student identity line

1.3 Vulnerability scanner (Tasks 2 and 6)

I created the official `vulnerability_scanner.py` (170 lines, 6,778 bytes). Run exactly as written it reported Struts and segmentation as **Unknown** and MySQL as **High**. The errors were environmental: `web_server` is a Docker-internal name that does not resolve from the Kali host, and the segmentation check crashed with a `utf-8` decode error on MySQL's binary reply. I made two minimal changes, shown in the diff: the Struts address became `http://127.0.0.1:8080`, and `errors="replace"` was added to the segmentation call. The instructor's detection logic was not changed. The re-run reported **Critical 1, High 1, Medium 1, Unknown 0**.

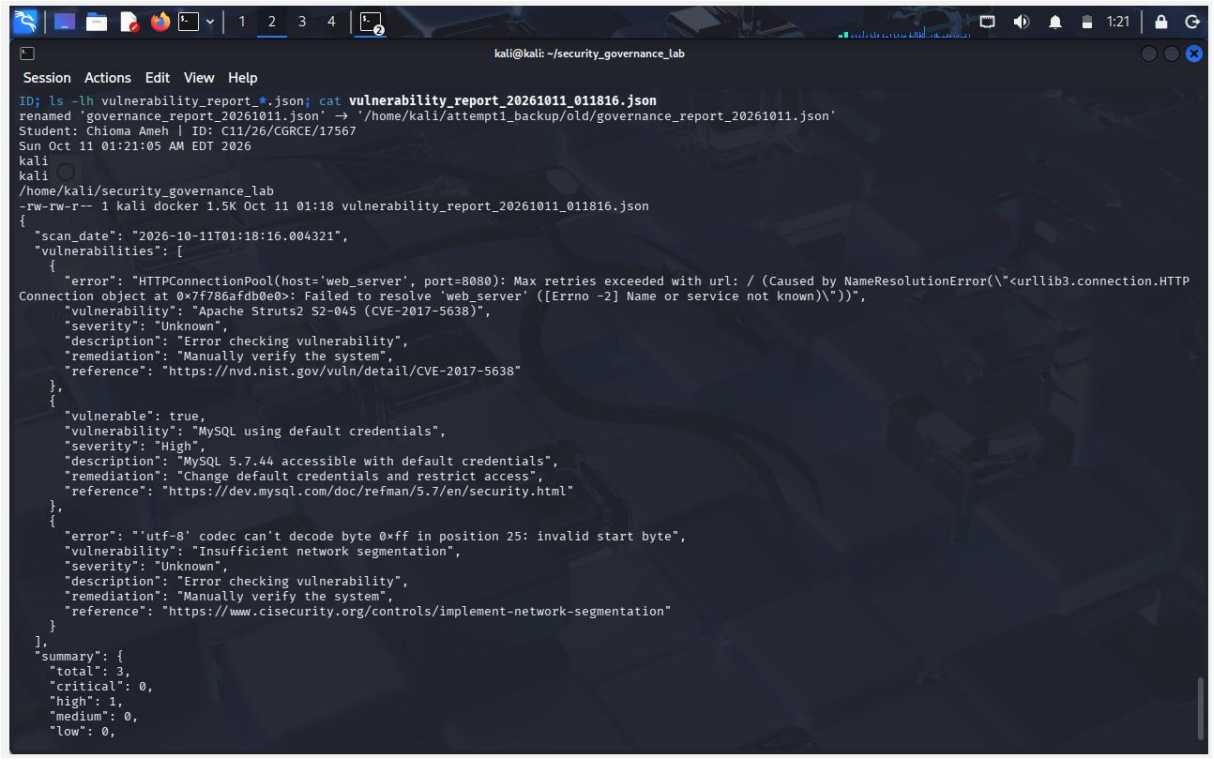


Figure 1.7 - Scanner as written: Struts and segmentation Unknown (NameResolutionError, utf-8 error), MySQL High

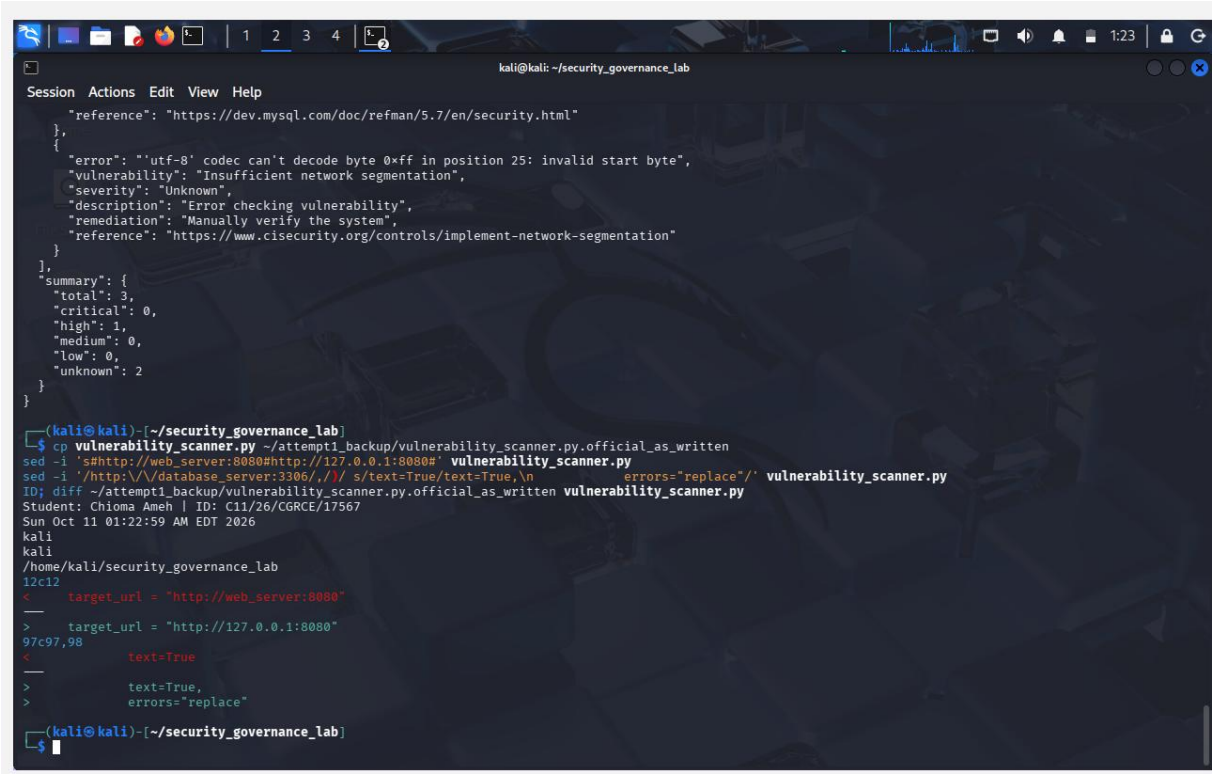


Figure 1.8 - Documented fix: diff of vulnerability_scanner.py (two changes)

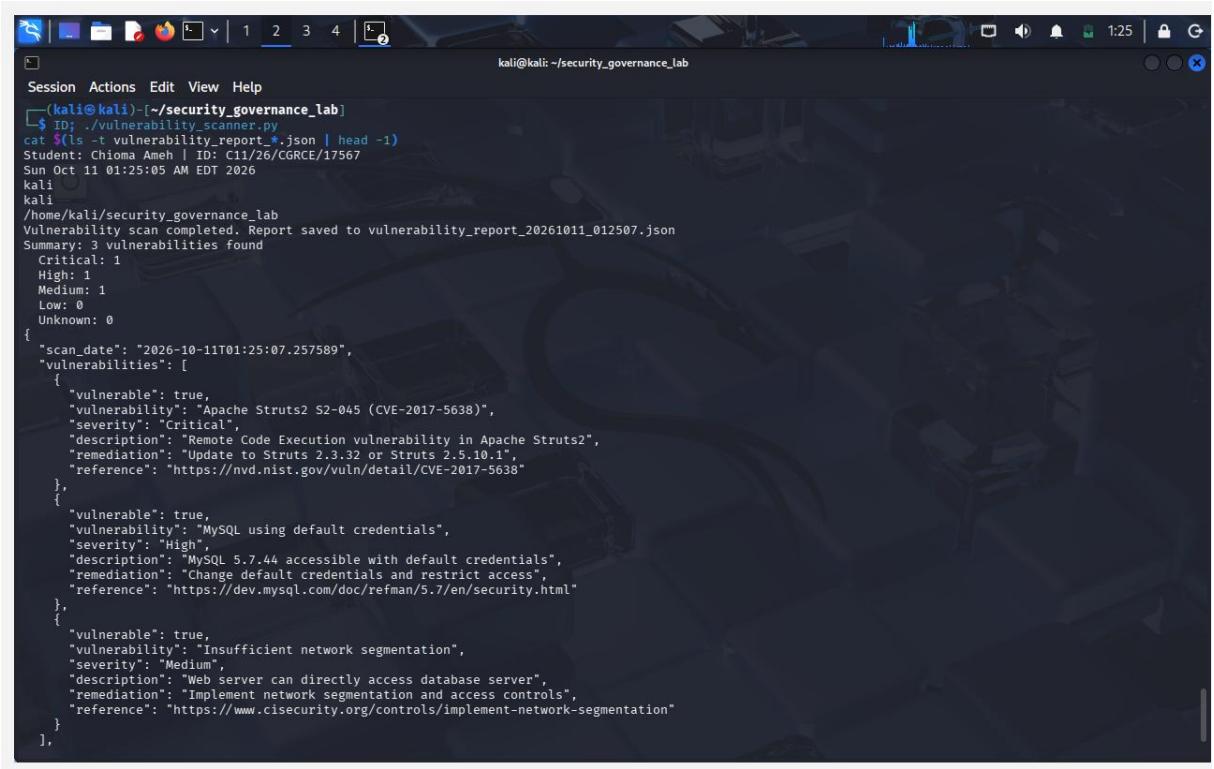


Figure 1.9 - Scan after the fix: Critical 1, High 1, Medium 1, Unknown 0

Finding	Severity	Scanner evidence
Apache Struts2 S2-045 (CVE-2017-5638)	Critical	Header returned by the crafted request: system vulnerable; remediation: update to 2.3.32 or 2.5.10.1
MySQL using default credentials	High	MySQL 5.7.44 accessible with root:password
Insufficient network segmentation	Medium	Web server can directly access the database server

1.4 Patch management and governance tracker (Tasks 3, 4 and 5)

I created the official `patch_management.py` and listed its three patches (PATCH-001 Struts, PATCH-002 MySQL, PATCH-003 segmentation); all showed status "Available" and none was applied in Part 1. I created the official `governance_tracker.py` (12 KB) and `initialize_governance.py`, which loaded the instructor's sample data. The tracker dashboard shows 3 policies, 3 controls, 3 risks (1 critical, 1 high, 1 medium), 1 incident, 3 metrics and 1 audit. The Patch Compliance metric is 85% against a 95% target and is shown as Red.

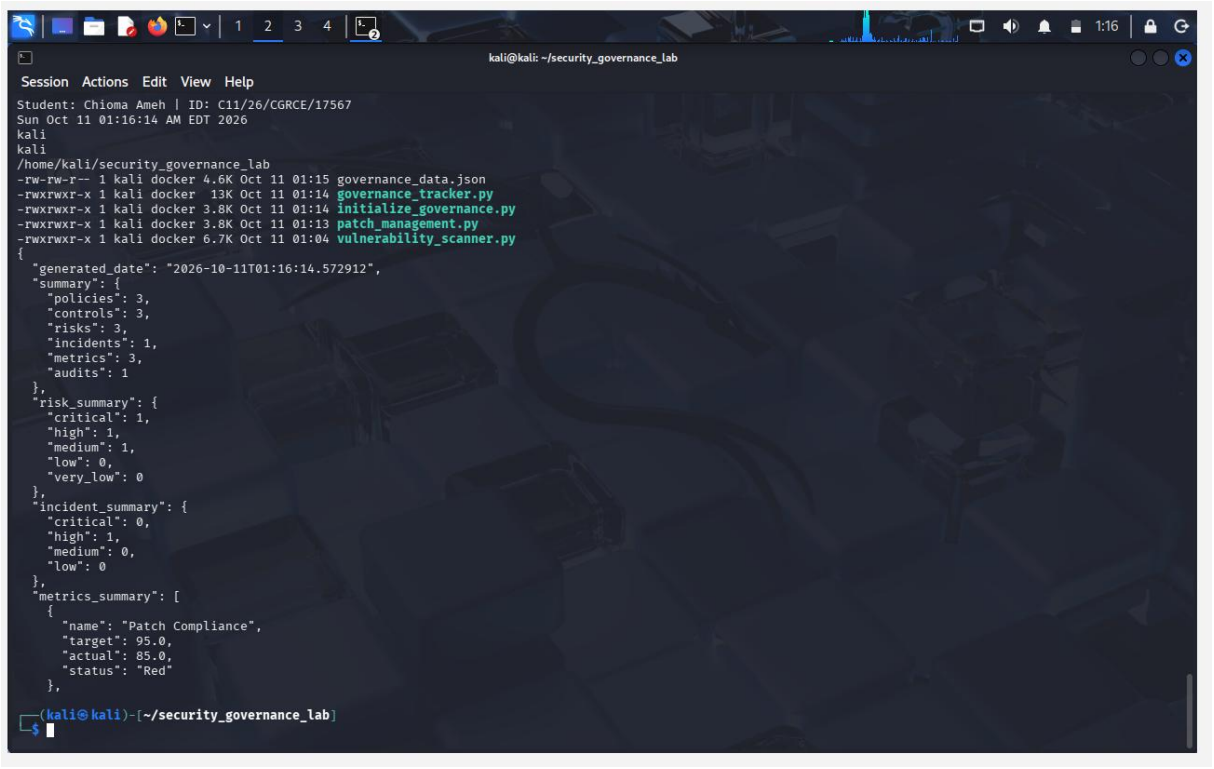


Figure 1.11 - Governance tracker dashboard after initialisation

1.5 Explanation: how the environment reflects the Equifax breach

The environment reproduces the conditions behind the 2017 Equifax breach. As at Equifax, the web tier ran a known-vulnerable Apache Struts version (CVE-2017-5638) for which a fix existed, and the scanner confirmed it as Critical. The database accepted default credentials, the web and database containers shared one flat network so a compromised web server could reach the data directly, and the monitoring container did nothing, so an intrusion would not be noticed.

The governance side matters as much as the technology. The tracker holds a patch management policy and a vulnerability scanning control on paper, yet patch compliance stands at 85% against a 95% target. That is the Equifax pattern: policies existed, but ownership, enforcement and verification failed. The lab therefore shows that these breaches come from process failures that technology merely exposes.

Part 2: Governance Failure Analysis

2.1 Attack simulation (Tasks 1-3)

The official `simulate_attack.py` sends a request with a crafted Content-Type header that asks Struts to add a harmless response header (X-Vulnerable). If that header comes back, the system is vulnerable and the script only **simulates** data access and exfiltration and writes a log; no real data is taken. Run as written it failed with the same `NameResolutionError` for `web_server`; I applied the same one-line address change as in the scanner (diff shown). It then reported that the system is vulnerable and wrote `attack_log.json` with `success: true` and `records_affected: 3`.

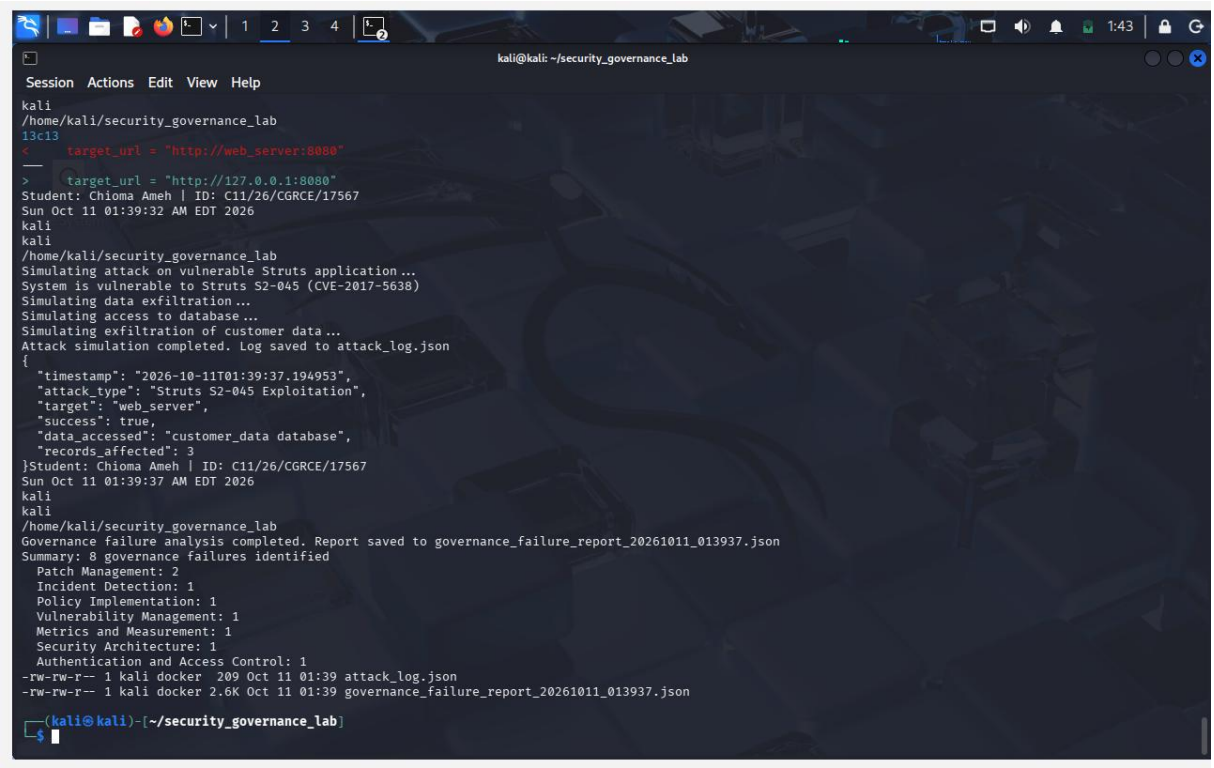


Figure 2.1 - Attack script as written (NameResolutionError), one-line fix, successful simulation and attack_log.json (simulated attack, success true, 3 records)

2.2 Governance failure analysis (Tasks 2 and 4)

The official `analyze_governance_failures.py` combined the scan report, the attack log, the tracker data and the patch status and identified **8 governance failures**.

Category (count)	What failed (technical)	Governance reading
Patch Management (2)	Struts unpatched (Critical); MySQL default credentials (High)	Known issues with fixes available were not remediated within a deadline
Incident Detection (1)	Simulated attack succeeded and nothing alerted	Monitoring existed in name only
Policy Implementation (1)	Patch policy exists but Struts is unpatched	Policy was not enforced or checked
Vulnerability Management (1)	Critical vulnerability open despite a policy	No owner or SLA enforcement
Metrics and Measurement (1)	Patch compliance metric Red but no remediation	Metrics did not drive action
Security Architecture (1)	Flat network between web and database	No segmentation standard

Category (count)	What failed (technical)	Governance reading
Authentication and Access Control (1)	Default database credentials	No credential standard or rotation

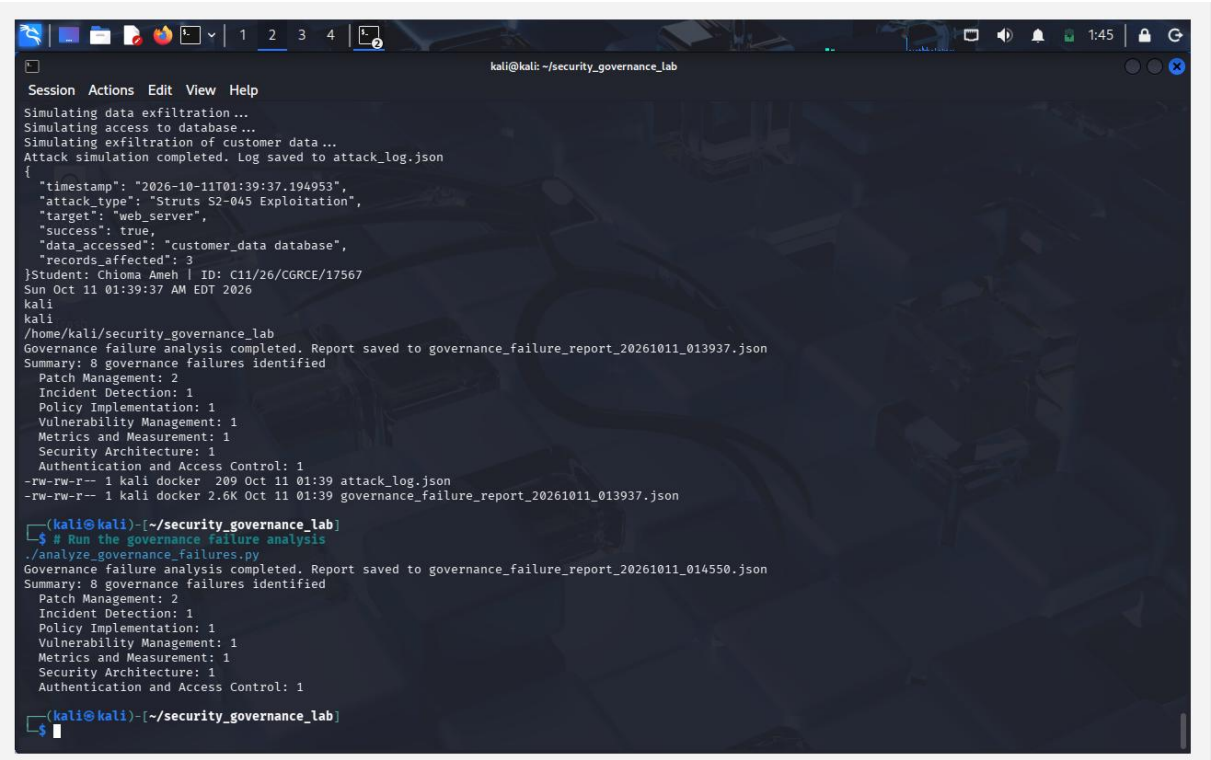


Figure 2.2 - Governance failure analysis: 8 failures by category

2.3 Comparison with the Equifax case study (Task 5)

`equifax_comparison.md` (44 lines, official) compares the lab with Equifax. I extend it here with the lab evidence and an ISO/IEC 27001:2013 Annex A reference for each failure.

Lab finding	Equifax parallel	Governance lesson	ISO 27001:2013 Annex A
Unpatched Struts	Known Struts flaw not patched after a fix was released in March 2017	Patching needs an owner, a deadline and verification	A.12.6.1 Management of technical vulnerabilities
Default MySQL credentials	Weak credentials and access controls	Credential standards are a governance responsibility	A.9.4.3 Password management system
Flat network	Attackers moved laterally	Segmentation contains a breach	A.13.1.3 Segregation in networks
No monitoring or alert	Breach undetected for about 76 days	Monitoring must be tested and owned	A.12.4.1 Event logging
Policy without enforcement	Policies not effectively implemented	A policy is only real if it is checked	A.5.1.1 Policies for information security
No clear ownership or board oversight	Unclear accountability and weak oversight	One accountable owner per risk; regular reporting	A.6.1.1 Information security roles and responsibilities

Where the lab differs from reality. The lab is a small simulation with three synthetic records; the "attack" only returns a header and writes a log, and Equifax was a far larger organisation. The lab demonstrates the pattern of failure, not its scale.

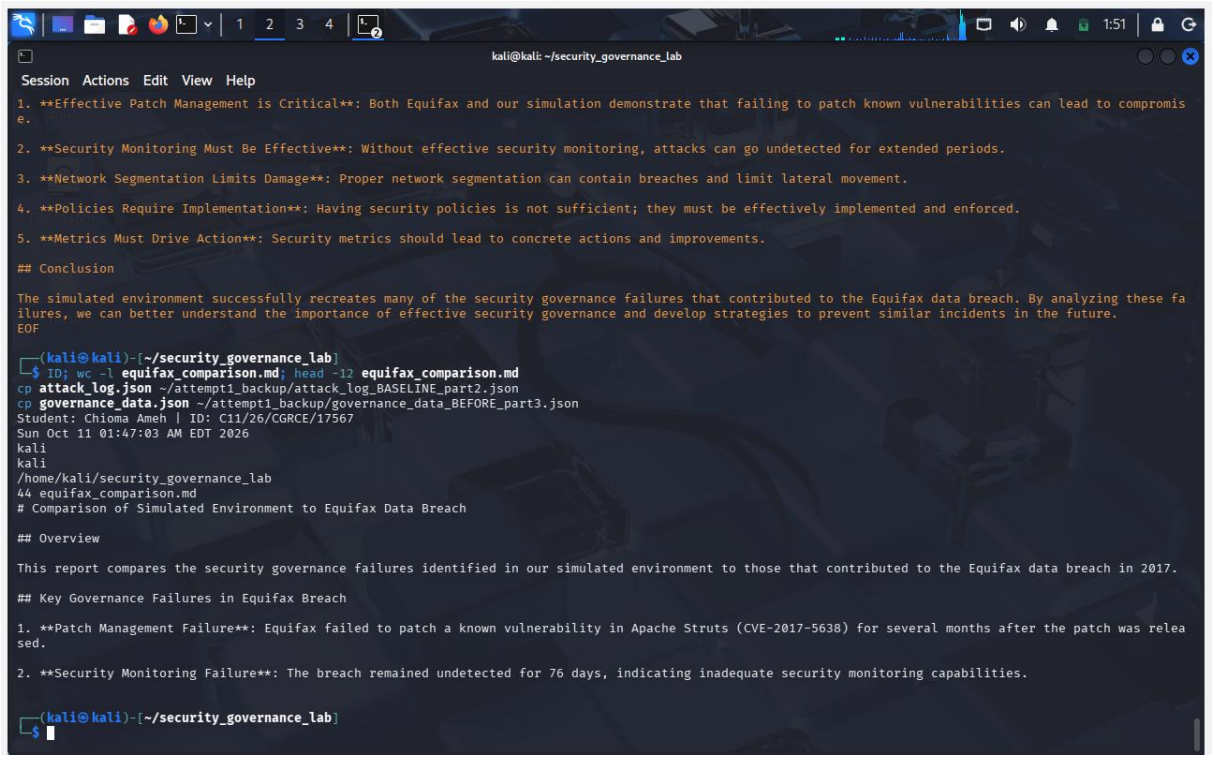


Figure 2.3 - equifax_comparison.md created (44 lines)

2.4 Explanation: key governance lessons

The most important lesson is that the technical weaknesses were all known and fixable: a published Struts fix, a default password and an unnecessary network path. They persisted because nobody was accountable for closing them, no deadline was enforced and nothing checked that fixes were applied. Chained together, an unpatched web server, a flat network and default database credentials let one flaw lead to the data, and with no monitoring nothing would reveal it.

The second lesson is that governance must verify, not just document. The lab's tracker shows policies and controls, but the metrics and scans show they were not working. Effective governance assigns ownership, sets deadlines, measures results and reports honestly to leadership. If I could change one thing to break the chain earliest, it would be enforcing a patch deadline with a named owner, because every later failure depended on the first.

Part 3: Governance Control Implementation

3.1 Official control implementation (Tasks 1-2)

I created the official `implement_governance_controls.py` (293 lines) and ran `--all` once. It reported five controls implemented.

Control	What the official script did	Tracker record
Patch management	Ran patch_management.py --apply PATCH-001 (writes a status file only)	CTL-004, MET-004
Network segmentation	Ran --apply PATCH-003 (writes a status file only)	CTL-005
Database security	Ran --apply PATCH-002 (writes a status file only)	CTL-006
Security monitoring	Wrote monitoring/security_monitor.sh and a cron entry file	CTL-007, MET-005
Governance oversight	Wrote governance_oversight.md (roles, board oversight, accountability)	POL-004, CTL-008

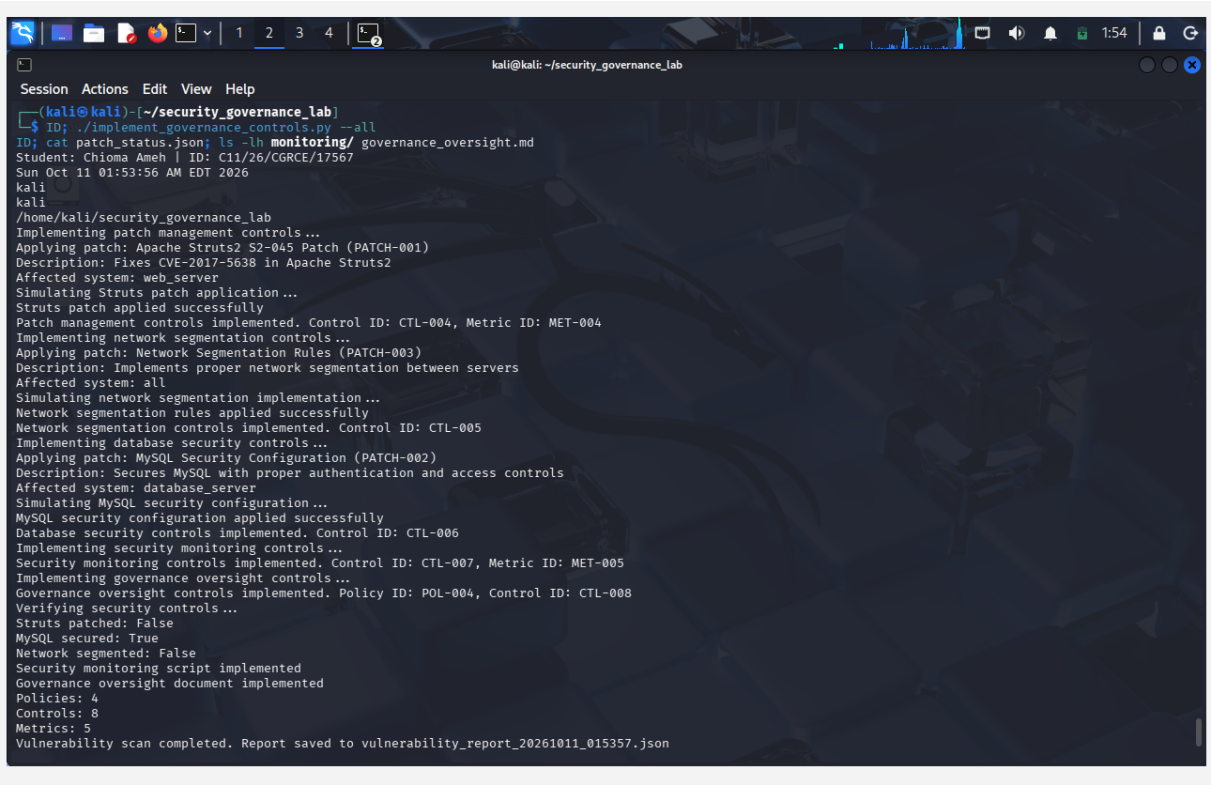


Figure 3.1 - Official implement_governance_controls.py --all: five controls implemented

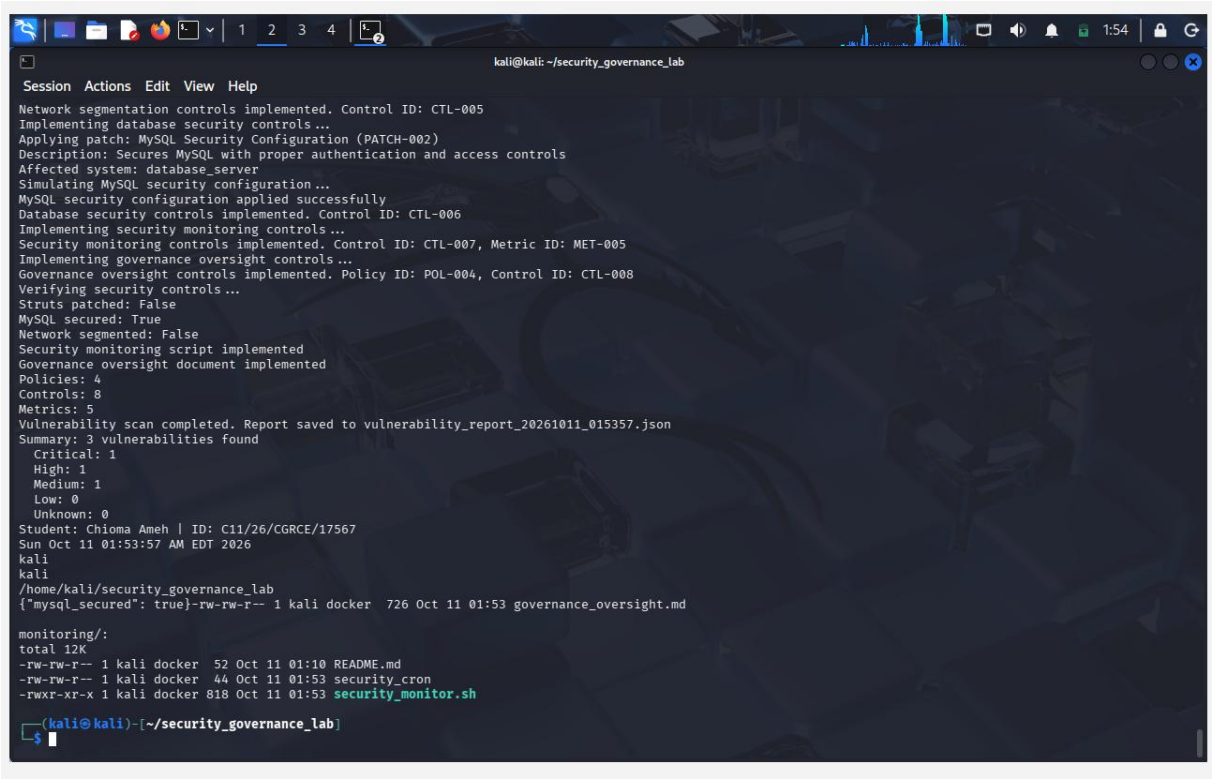


Figure 3.2 - patch_status.json, monitoring files and governance_oversight.md created

3.2 Verification as written, and what it revealed

I created `governance_controls_report.md` (87 lines) and ran the instructor's verification steps. The results contradict the claims in the official report:

Verification item	Result (before supplementary remediation)
Struts patched / MySQL secured / Network segmented	False / True / False (each simulated patch overwrote the previous status file)
Policies / controls / metrics in tracker	4 / 8 / 5 (up from 3 / 3 / 3)
Re-scan after controls	Critical 1, High 1, Medium 1 (unchanged)
Attack simulation re-run ("verify it fails")	Still vulnerable; attack succeeded

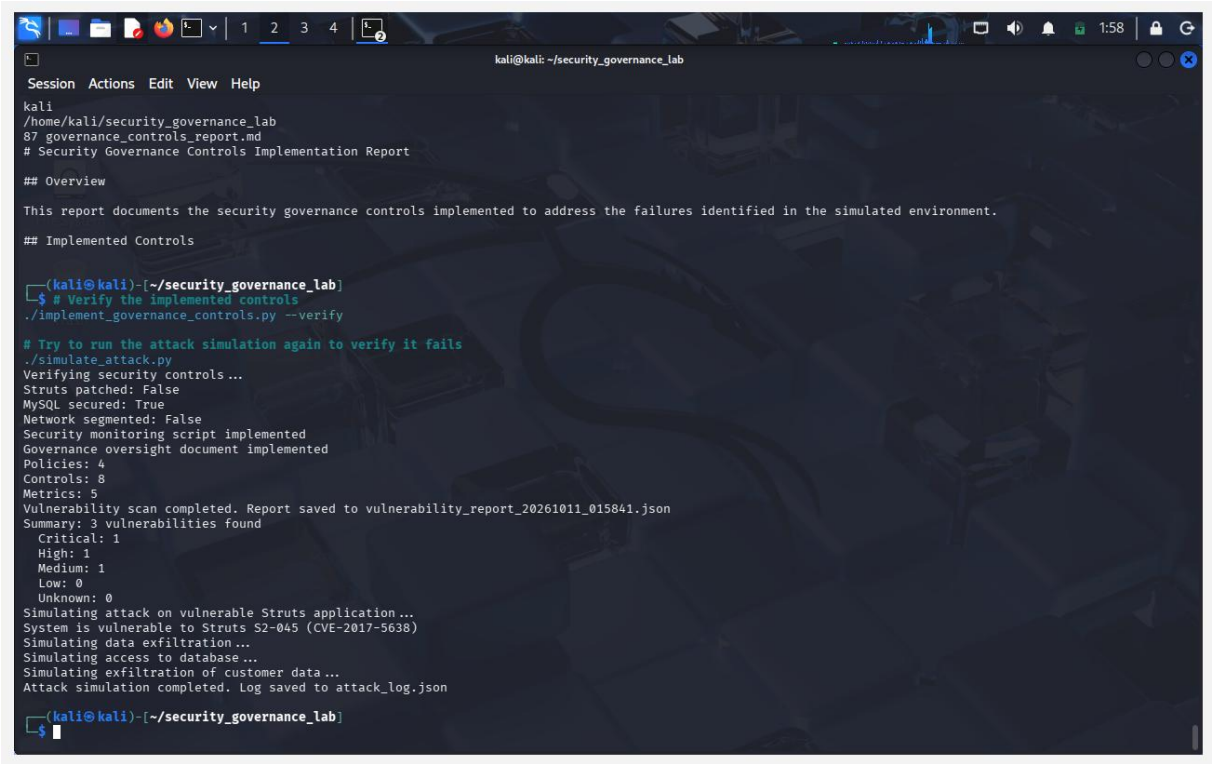


Figure 3.3 - Verification as written: status lines, unchanged scan, attack still succeeds

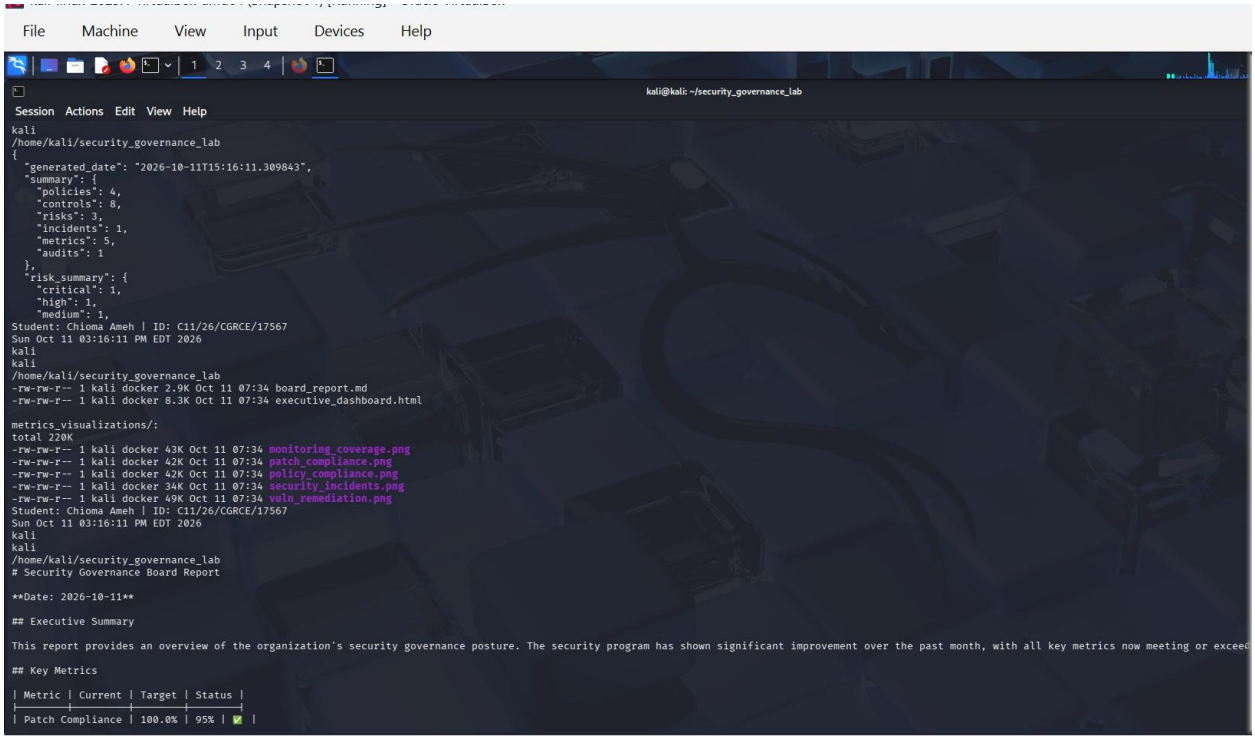


Figure 3.4 - Tracker dashboard after the official controls: 4 policies, 8 controls, 5 metrics; risk register still 1 critical, 1 high, 1 medium

Finding. The governance layer changed (policies, controls and metrics increased, and documents were created), but the technical risk did not: the official "patches" write a status file and never change the containers, and each patch overwrites the previous one. This is itself a lesson from the Equifax case: controls recorded on paper are not controls that operate. The official controls report states that the controls were verified; my evidence shows that statement was not supported until the supplementary remediation below.

3.3 Supplementary real remediation

To obtain genuine verification evidence I performed the actions the official script comments describe ("in a real environment, this would update..."), using only standard Docker and MySQL commands, with no new scripts and no changes to official files. This section is clearly labelled supplementary.

Control	Action	Evidence
Database security	Replaced the default root and app_user passwords with random lab-only passwords stored in a private file outside the repository	Old password: ERROR 1045 Access denied, exit code 1
Network segmentation	Created an internal backend_network; moved database_server onto it and off secgov_network	database_server alone on backend_network (internal=true); web-to-database wget failed, exit code 4
Patch management	Replaced the vulnerable container with vulhub/struts2:2.3.34-showcase, a release after the 2.3.32 fix; bound the port to 127.0.0.1	docker ps shows the new image; scanner: Struts not vulnerable; attack script: not vulnerable

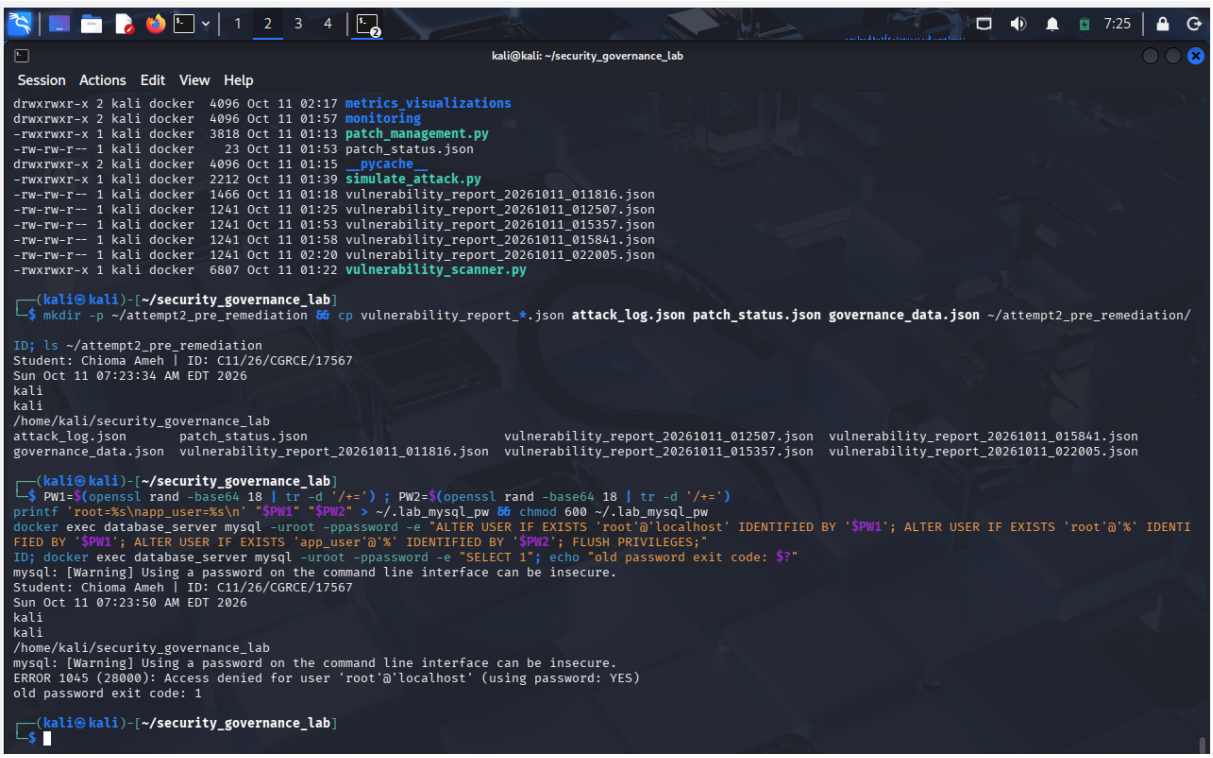


Figure 3.5 - Database: default password rejected (Access denied, exit code 1)

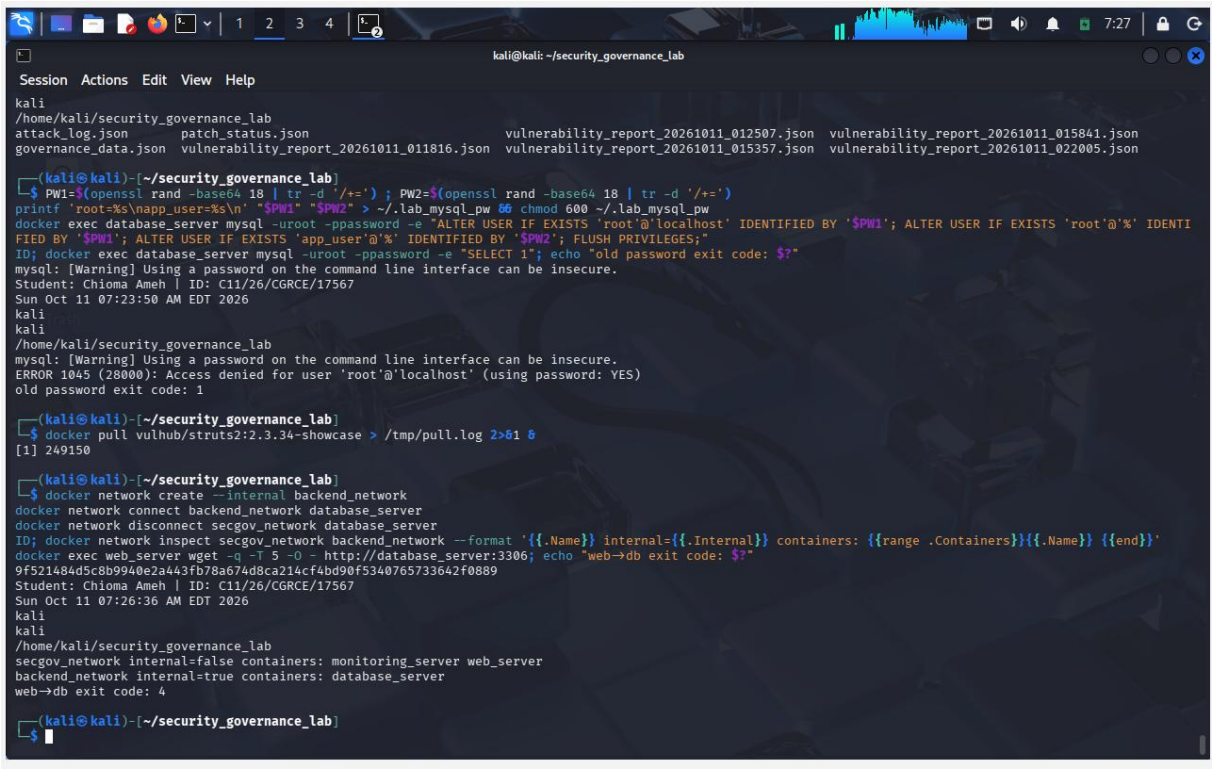


Figure 3.6 - Segmentation: database on its own internal network; web-to-database access fails



Figure 3.7 - After remediation: web_server runs vulhub/struts2:2.3.34-showcase on 127.0.0.1:8080; database_server is alone on backend_network (internal=true)



Figure 3.8 - Official scanner after remediation: Critical 0, High 0, Medium 1, Unknown 0 (vulnerability_report_20261011_154310.json)



Figure 3.9 - Official attack simulation after remediation: system not vulnerable to Struts S2-045

	Before	After	Result
Struts S2-045	Critical, vulnerable	Not vulnerable	Fixed
MySQL default credentials	High, accessible	Not accessible	Fixed
Network segmentation	Medium	Medium (flagged by the official check)	Fixed in practice; manual test shows no access

	Before	After	Result
Scan summary	Critical 1, High 1, Medium 1	Critical 0, High 0, Medium 1	Improved
Attack simulation	Vulnerable	Not vulnerable	Blocked

Why Medium remains. The official segmentation check reports "vulnerable" whenever the text `DOCTYPE HTML` is missing from the output. A database port never returns that text, so the check cannot report success whatever the network looks like. My evidence for segmentation is therefore the Docker network listing and the failed web-to-database connection, not the scanner line.

3.4 Explanation: how the controls address the Equifax failures

Each control answers a specific failure. Upgrading Struts closes the same kind of hole Equifax left open, and the tracker's patch policy and SLA provide the process that was missing. Replacing default database credentials and putting the database on a separate internal network limits what a compromised web server can reach, which addresses the weak credentials and lateral movement at Equifax. The monitoring script and the oversight document describe what should detect and govern, and the tracker adds owners and metrics.

The verification showed why proof matters: the first set of controls looked complete on paper but changed nothing, so verification caught a gap that a paperwork-only review would have missed.

3.5 Residual risks and limitations

- The monitoring script reads an nginx log and uses a MySQL client that do not exist in the monitoring container and is not scheduled to run, so detection is designed but not operating. The attack simulation returns a header; it does not exercise real monitoring.
- The Struts image is still a deliberately vulnerable lab image for other flaws; I only showed that the specific scanned flaw is remediated.
- The official status file holds one key at a time, so it cannot show all three patches as applied.
- The tracker's sample data and the controls report text are the instructor's and were not generated from my measurements.

Part 4: Metrics and Executive Reporting

4.1 Deliverables produced

I created the official `governance_metrics.py` (466 lines) and ran it. It generated `executive_dashboard.html`, `board_report.md` and five charts in `metrics_visualizations/`. I also created `governance_metrics_framework.md` and `dashboard_user_guide.md`. I re-ran the metrics script after remediation so the dashboard is dated after the fixes.

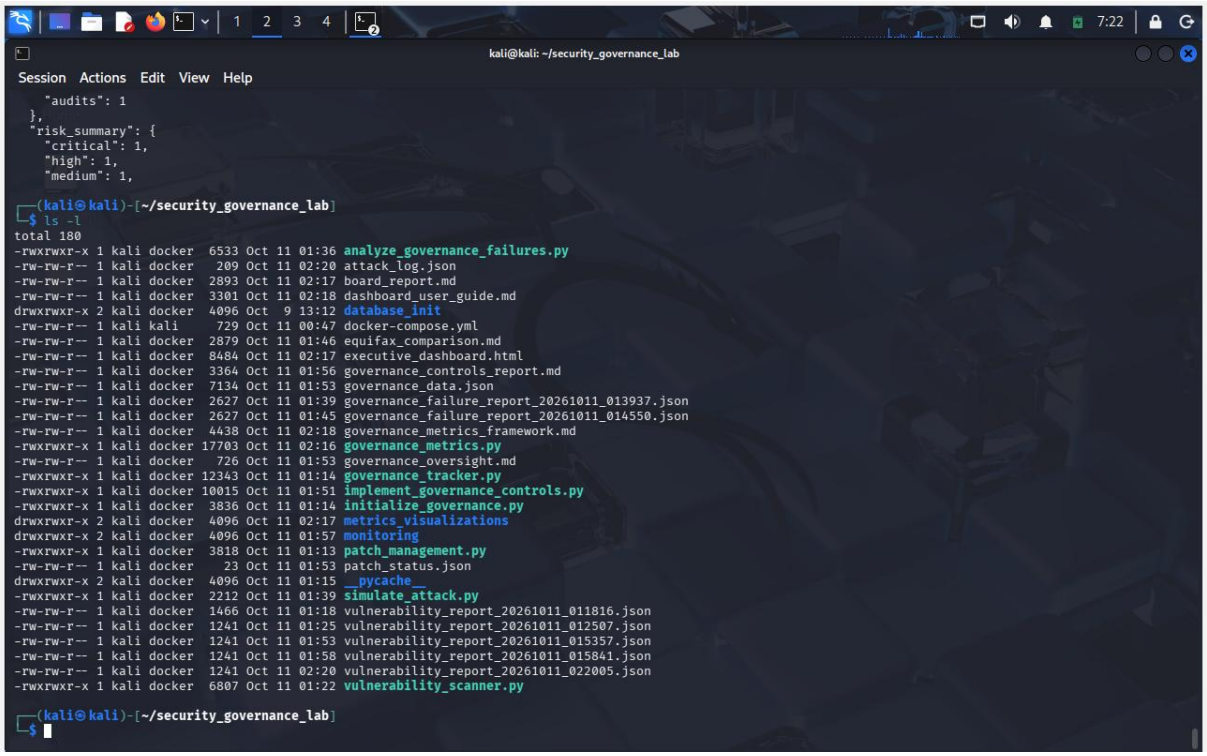


Figure 4.1 - Part 4 files created (ls -lh, sizes)

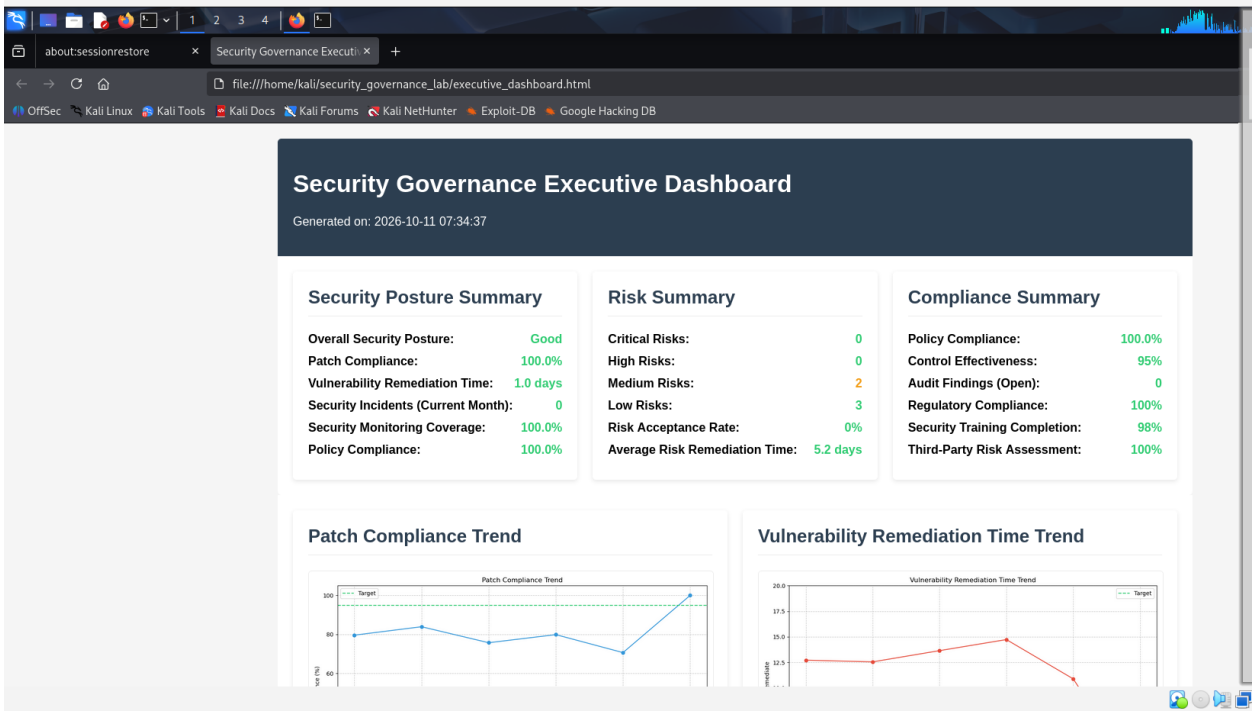


Figure 4.2 - Executive dashboard in Firefox (generated 2026-10-11 07:34:37). Several values are fixed in the official script; see the data provenance section

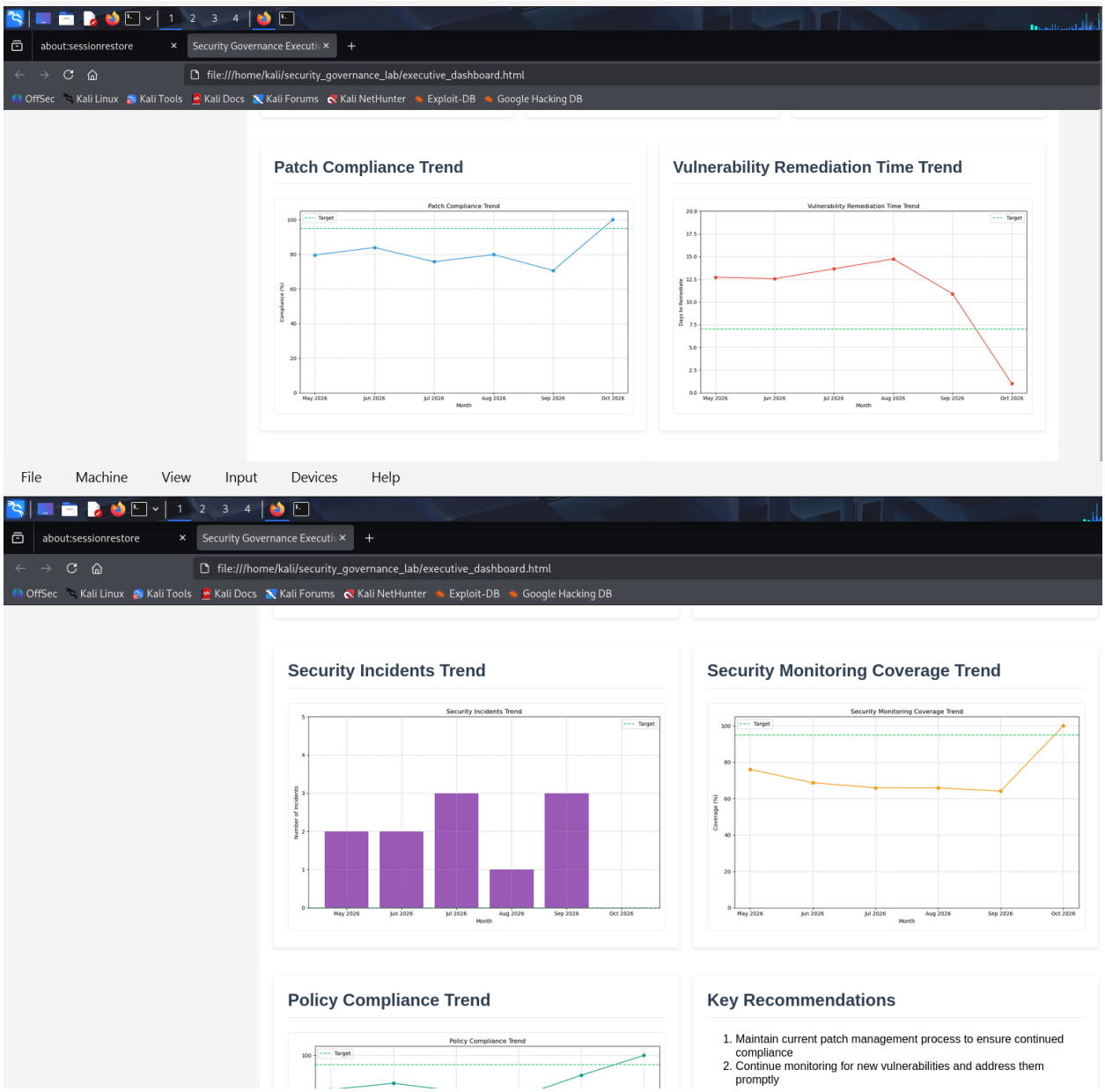


Figure 4.3 - Dashboard trend charts: patch compliance, remediation time, security incidents, monitoring coverage (history generated with random values)

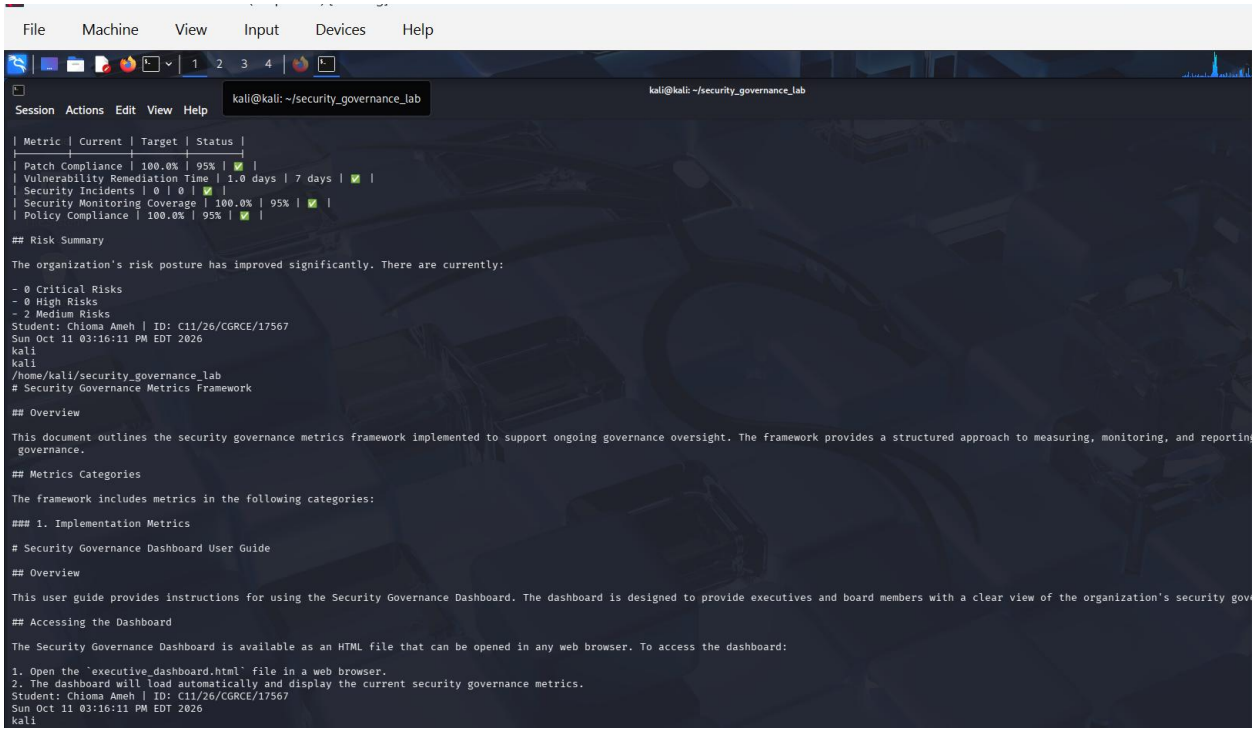


Figure 4.4 - Board report: key metrics table and risk summary (the official script's fixed sample values), with student identity line

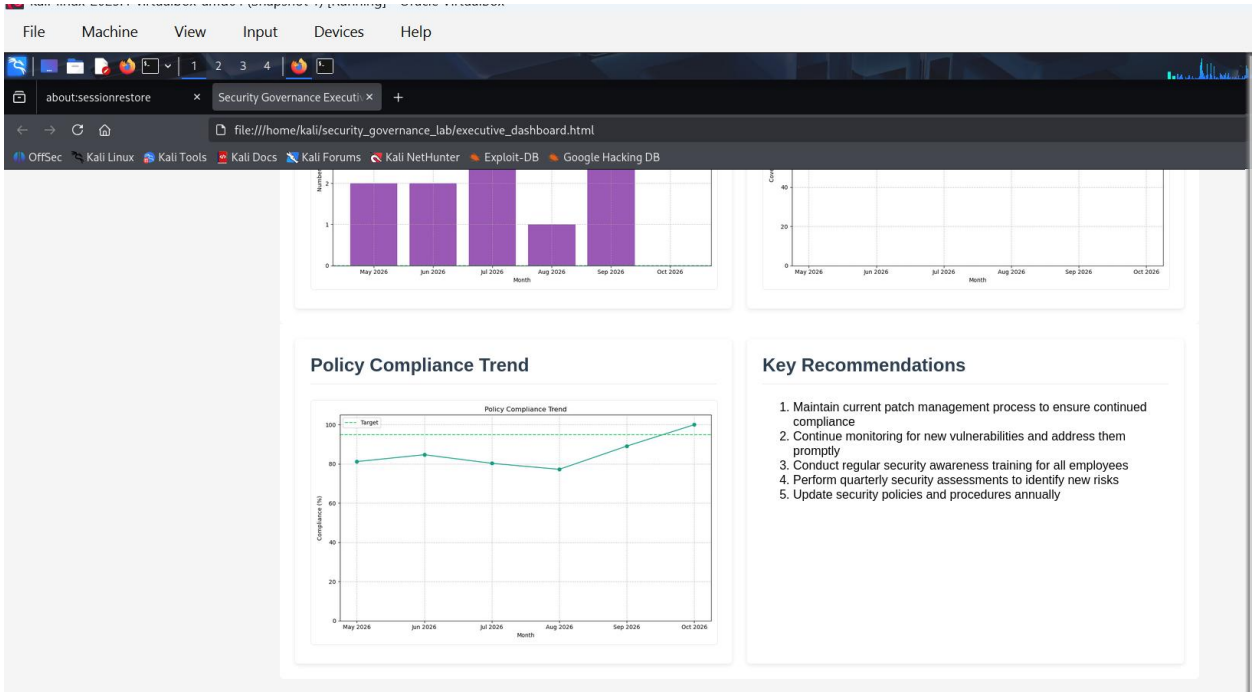


Figure 4.5 - Dashboard: policy compliance trend chart and key recommendations

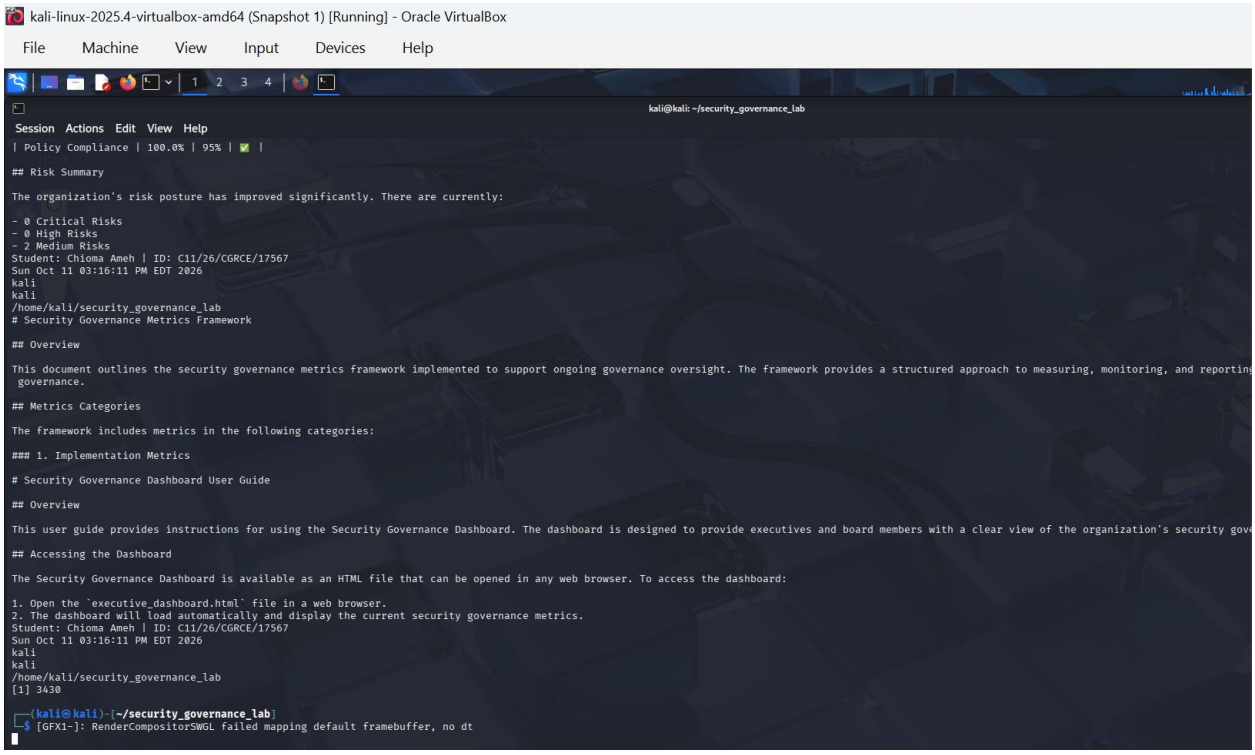


Figure 4.6 - Metrics framework and dashboard user guide files (head of each), with student identity line

4.2 Data provenance: what the dashboard shows versus what I measured

Reading `governance_metrics.py` shows that the six-month history is generated with `random.uniform` and `random.randint`, and several dashboard values are fixed in the code ("Overall Security Posture: Good", 0 critical and 0 high risks, 0 incidents, 100% compliance, 95% control effectiveness). They do not come from my scans, so I treat the dashboard as an illustrative template and report the measured values separately.

Official dashboard/board report says	My measured evidence	Comment
Overall posture: Good	Before remediation: Critical 1, High 1, Medium 1; after: 0 / 0 / 1	Posture was not good until real remediation
Patch compliance 100%	Official verify: Struts patched False; scan Critical 1 (before remediation)	Hard-coded value, not measured
0 incidents this month	Attack simulation succeeded before remediation; the tracker lists 1 incident	Contradicted by the lab's own attack log
0 critical and 0 high risks	Tracker risk summary: 1 critical, 1 high, 1 medium	The dashboard's fixed 0 critical / 0 high does not reflect the tracker's risk register, which still lists 1 critical and 1 high risk
Control effectiveness 95%	After remediation the scanner cleared 2 of 3 checks; the third is cleared by manual test	Hard-coded value, not measured
Trend charts	Random values on each run	Illustrative only; do not use as evidence

4.3 Measured governance metrics from this lab

Metric	Baseline	After controls	Source
Open Critical + High findings	2	0	Scanner (before / after remediation)
Scanner checks clear	0 of 3	2 of 3 (3 of 3 with manual proof)	Scanner + manual test
Attack simulation succeeds	Yes	No	simulate_attack.py
Policies / controls / metrics tracked	3 / 3 / 3	4 / 8 / 5	governance_tracker.py
Controls operating (verified by test)	0 of 3 technical controls	3 of 3 technical controls	Supplementary remediation
Detection of the simulated attack	No alert	No alert (monitoring not operating)	Residual risk

The framework document groups metrics into implementation, effectiveness, efficiency and impact categories and defines each metric by name, formula, target, data source, frequency, owner and audience. The user guide explains how to read the colour status indicators and how to refresh the dashboard. In a real organisation each metric would be computed from live scan and ticket data rather than random values.

4.4 Explanation: how metrics and reporting support governance

Metrics turn technical findings into information leaders can act on. A board cannot read scan output, but it can ask whether risk is falling, who owns each risk, whether deadlines are met and whether controls work when tested. A dashboard and board report give that view each month and create a record of decisions.

The lab also showed the risk of reporting without evidence. The official dashboard says everything is good while my scans and tests show otherwise. Good governance reporting shows its data source, flags what is simulated and reconciles it with measured results, as in section 4.2.

Conclusion and limitations

The lab reproduced the main governance failures of the Equifax breach in miniature and showed how they chain together. It also showed that controls must be verified: the official controls were recorded but not operating, and only a real remediation changed the scan and attack results. The main limitations are that this is a small simulation with synthetic data, the attack is simulated, the monitoring control is not operating, the official segmentation check cannot clear, and the official dashboard values are not measured.

References

- US House Committee on Oversight and Government Reform, "The Equifax Data Breach", majority staff report, December 2018.
- US Senate Permanent Subcommittee on Investigations, "How Equifax Neglected Cybersecurity and Suffered a Devastating Data Breach", 2019.
- US Government Accountability Office, GAO-18-559, "Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach", 2018.
- NIST National Vulnerability Database, CVE-2017-5638 (Apache Struts 2).
- Apache Struts security bulletin S2-045.
- US Federal Trade Commission, Equifax settlement announcement, July 2019.
- ISO/IEC 27001:2013, Annex A controls.
- ICDFA GRC102 Practical Lab 8 instructions (LAB_INSTRUCTIONS.md) and assessment brief.

Appendix A: Troubleshooting log and deviations

#	Problem	Cause	Fix	Evidence
1	docker: command not found	Docker not installed on Kali	sudo apt install docker.io docker-compose	Fig 1.2; error screenshot Screenshot_2026-10-09_165046 in the evidence folder
2	permission denied on /var/run/docker.sock	User not in the docker group yet	usermod -aG docker; newgrp docker	Screenshot 2026-10-09_180922 / _181242
3	manifest for vulhub/struts2:s2-045 not found	Docker Hub no longer serves that tag; same image stored locally as 2.3.30	docker tag vulhub/struts2:2.3.30 vulhub/struts2:s2-045 (compose file unchanged)	Fig 1.5
4	Scanner: Struts Unknown, NameResolutionError web_server	web_server resolves only inside Docker; scanner runs on the host	Changed address to http://127.0.0.1:8080	Figs 1.7, 1.8
5	Scanner: segmentation Unknown, utf-8 codec error (0xff)	Python could not decode MySQL's binary reply	Added errors="replace"	Figs 1.7, 1.8
6	simulate_attack.py: NameResolutionError	Same hostname issue	Same one-line address change	Fig 2.1
7	First attempt used shortened scripts	I wrote condensed versions	Replaced with the official scripts; first attempt kept in evidence folder	evidence/attempt1_first_run
8	Official verification contradicts the controls report	Patches write a status file only and overwrite each other	Documented as a finding; supplementary real remediation	Figs 3.3-3.9
9	Matplotlib/pyparsing warnings	Library message, not an error	None needed	Part 4 output

Appendix B: Repository file inventory

File / folder	Purpose (brief deliverable)
docker-compose.yml, database_init/init.sql	Environment definition and database initialisation
vulnerability_scanner.py, vulnerability_report_*.json	Scanner and scan reports (before and after fixes)
patch_management.py, governance_tracker.py, initialize_governance.py, governance_data.json	Patch system, tracker and tracker data
simulate_attack.py, attack_log.json	Attack simulation and log
analyze_governance_failures.py, governance_failure_report_*.json	Governance failure analysis and reports
equifax_comparison.md	Equifax comparison report
implement_governance_controls.py, governance_controls_report.md, patch_status.json, governance_oversight.md, monitoring/	Control implementation script, report and outputs
governance_metrics.py, executive_dashboard.html, board_report.md, metrics_visualizations/	Metrics script, dashboard, board report, charts
governance_metrics_framework.md, dashboard_user_guide.md	Metrics framework and dashboard user guide
evidence/, screenshots/, README.md	Troubleshooting evidence, labelled screenshots and repository guide